

IES FRANCISCO DE GOYA – LA ELIPA DEPARTAMENTO DE INFORMÁTICA				Dirección del Área Territorial de Madrid Capital CONSEJERÍA DE EDUCACIÓN, JUVENTUD Y DEPORTE Comunidad de Madrid 
Titulación:	DAM	Curso:	2025/2026	
Módulo:	Bases de Datos	Fecha:	27 de abril de 2026	
Examen:	UT7: Programación de BBDD	Duración:	50 minutos	
Nombre:		Calificación:		
Apellidos:				
RAs:	RA5: Desarrolla procedimientos almacenados evaluando y utilizando las sentencias del lenguaje incorporado en el sistema gestor de bases de datos.			

Instrucciones

- Los ejercicios deben resolverse utilizando sintaxis estándar de MySQL.
- Se valorará la limpieza del código, el uso de delimitadores y la correcta definición de tipos.
- La base de datos de referencia para todos los ejercicios es **Sakila**.

Ejercicio 1 (4 puntos) – (RA5-CE.g) Procedimientos con bucles

Crea un procedimiento almacenado llamado `p_bulk_add_inventory` que permita añadir varias copias de una misma película a una tienda específica de forma automática. El procedimiento debe cumplir:

- Recibirá cuatro parámetros: `p_film_id`, `p_store_id`, `p_quantity` y `p_last_id`.
- Debe contener una estructura **IF** que valide que la cantidad sea mayor que 0.
- Utilizará un bucle **WHILE** para realizar las inserciones en la tabla `inventory`.
- Al finalizar, el parámetro de salida `p_last_id` debe contener el ID del último registro insertado.

Ejemplo de ejecución:

```
CALL p_bulk_add_inventory(1, 1, 3, @ultimo_id);
SELECT @ultimo_id;
```

Resultado esperado: Se insertarán 3 filas en la tabla `inventory`. El SELECT mostrará el ID **4584** (asumiendo que partimos del inventario original de Sakila).

Pista: Para obtener el ID generado por el último INSERT en una columna autoincremental, utiliza la función de sistema LAST_INSERT_ID().

Ejercicio 2 (3 puntos) – Triggers

Implementa un trigger llamado `tr_check_payment_limit` que actúe sobre la tabla `payment` para evitar errores tipográficos en los pagos manuales.

- El trigger debe ejecutarse **antes de insertar** un nuevo registro.
- Si el importe del pago (`amount`) es superior a **100.00**, el trigger debe cancelar la operación y mostrar un mensaje de error: «Error: El importe excede el límite de seguridad de 100 euros».

Ejemplo de ejecución:

```
INSERT INTO payment (customer_id, staff_id, rental_id, amount,
    payment_date)
VALUES (1, 1, 1, 150.00, NOW());
```

Resultado esperado: ERROR 1644 (45000): Error: El importe excede el limite de seguridad de 100 euros.

Ejercicio 3 (3 puntos) – Funciones

El departamento de marketing quiere realizar una campaña para recuperar clientes antiguos, pero necesitan saber cuánto tiempo lleva un cliente sin que se actualice su ficha en nuestro sistema.

Desarrolla una función llamada `f_days_inactive` que reciba el código de un cliente y nos devuelva un número entero con el total de días transcurridos desde su última actualización (`last_update`) hasta el momento actual.

Ejemplo de ejecución:

```
SELECT first_name, f_days_inactive(customer_id) FROM customer WHERE
    customer_id = 1;
```

Resultado esperado: Una fila con el nombre **MARY** y el valor **7373** (días transcurridos desde 2006 hasta hoy).

Nota: No olvides marcar la función como `NOT DETERMINISTIC` ya que utiliza `NOW()`.